マカロに対する定数回シャッフルのゼロ知識証明プロトコル

松村 隆誠[1,a)]　品川 和雅[1,2,b)]

概要：パズルに対するカードベースゼロ知識証明プロトコルとは、パズルの解を知っている証明者が、検証者に解に関する情報を一切明かすことなく解が存在することを、カードを用いて納得させるプロトコルである。マカロは、部屋条件・隣接条件・矢印条件と呼ばれる 3 つの条件を満たすように数字を配置するペンシルパズルである。2018 年に Bultel らはマカロに対するカードベースゼロ知識証明プロトコルを提案した。彼らのプロトコルは、盤面サイズや部屋の大きさに応じて必要なシャッフル回数が増加するという特徴があった。本稿では、マカロに対するシャッフル 3 回のプロトコルとシャッフル 2 回のプロトコルを提案する。前者は Bultel らと同じ入力方式であり、後者は座標情報を入力する方式である。また、シャッフルの種類はいずれもパイルスクランブルシャッフルである。

キーワード：カードベース暗号、ゼロ知識証明、マカロ

Constant-Shuffle Zero-Knowledge Proof Protocols for Makaro

Ryusei Matsumura[1,a)]　Kazumasa Shinagawa[1,2,b)]

Abstract: A card-based zero-knowledge proof protocol for a puzzle enables a prover, who knows a solution to the puzzle, to convince a verifier that a valid solution exists without revealing any information about the solution, using only cards. Makaro is a pencil puzzle in which numbers must be placed so as to satisfy three conditions: the room condition, the adjacency condition, and the arrow condition. In 2018, Bultel et al. proposed a card-based zero-knowledge proof protocol for Makaro. Their protocol requires a number of shuffles that increases with the board size and the maximum room size. In this paper, we propose a three-shuffle protocol and a two-shuffle protocol for Makaro. The former uses the same encoding as Bultel et al., whereas the latter adopts an encoding based on coordinate information. Both protocols employ pile-scramble shuffles.

Keywords: Card-based cryptography, Zero-knowledge proofs, Makaro

1. はじめに

1.1 マカロ

マカロは、パズル通信ニコリによって発表されているペンシルパズルである [6]。マカロの盤面は、いくつかの「部屋」に分割された白マスと、黒マスから構成される。また、いくつかの黒マスには、その黒マスに隣接する白マスのいずれかの方向を指し示す矢印が描かれている。プレイヤーの目的は、以下の 3 つの条件をすべて満たすように、各白マスに数字を割り当てることである。

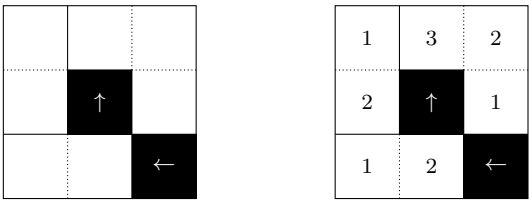
- **部屋条件**　それぞれの部屋に、1 からその部屋のマス数までの数字を 1 つずつ割り当てる。
- **隣接条件**　縦または横に隣接する 2 つの白マスには、同じ数字を割り当ててはならない。
- **矢印条件**　矢印が描かれた各黒マスについて、その上下左右に隣接する白マスのうち、矢印が指す白マスに単独で最大の数字を割り当てる。

すべての白マスへの数字の割当てが上記の条件を満たすとき、その割当てをマカロの解と呼ぶ。図 1 に、マカロの問題例とその解の例を示す。

[1] 筑波大学 University of Tsukuba
[2] 産業技術総合研究所 National Institute of Advanced Industrial Science and Technology
[a)] s2310483@u.tsukuba.ac.jp
[b)] shinagawa@cs.tsukuba.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.



(a) 問題例　　(b) 解答

図 1: パズルの例題と解答

1.2 ゼロ知識証明

ゼロ知識証明 [2] とは、証明者 P がある命題が正しいことを、命題が正しいことを示す証拠の情報を明らかにすることなく検証者 V に納得させる暗号技術である。

ゼロ知識証明では、完全性、健全性、ゼロ知識性の 3 つの性質を満たすことが求められる。

- **完全性** 命題が正しいとき、証明者 P と検証者 V が正しくプロトコルを実行した場合、必ず受理される。
- **健全性** 命題が正しくないとき、証明者 P がどのように振る舞ったとしても、必ず拒否される。
- **ゼロ知識性** 検証者 V は、その命題が正しいという事実以外の情報を得ることができない。

また、物理的なカードを用いてゼロ知識証明を構成する手法は、カードベースゼロ知識証明と呼ばれる。この手法では、カードの裏返しやシャッフルなどの物理操作のみを用いてプロトコルを構成するため、直感的な検証が可能となる。

1.3 貢献

本稿では、マカロに対するカードベースゼロ知識証明を対象とする。既存研究では Bultel ら [1] によってマカロに対するカードベースゼロ知識証明プロトコルが提案されているが、必要なシャッフル回数は盤面サイズに依存する。本稿では、定数回のシャッフルを用いる 2 種類のプロトコルを提案する。既存手法と提案手法におけるカード枚数およびシャッフル回数の比較を表 1 に示す。

提案プロトコル 1 は 3 回、提案プロトコル 2 は 2 回のパイルスクランブルシャッフルを用いる。これらのプロトコルのカード枚数は同じであるが、証明者 P が配置する入力カード列の置き方に違いがある。提案プロトコル 1 では、各白マスに整数を配置するが、提案プロトコル 2 では各白マスに行番号と列番号を配置する。本稿では、前者を標準入力、後者を座標入力と呼ぶ。このように、入力カード列の置き方のモデルが異なるため、提案プロトコル 2 が提案プロトコル 1 と比べて真に優れているわけではない。

表 1: マカロに対するゼロ知識証明プロトコルの比較

手法	カード枚数	シャッフル回数
Bultel ら [1]	$kn+5k-5$	$2(t+l+a)$
提案プロトコル 1	$(4k+3)n+8a(k^2+k-1)+5kt$	3
提案プロトコル 2	$(4k+3)n+8a(k^2+k-1)+5kt$	2

n は白マスの個数、k は部屋サイズの最大値、l は部屋の個数、a は矢印の個数、t は異なる部屋に属する隣接マスの組数を表す。

2. 準備

2.1 使用するカード

- **二色カード** $\boxed{\heartsuit}\boxed{\clubsuit}$: 二色カードは、表面にハートまたはクラブが書かれたカードである。
- **数字カード** $\boxed{1}\boxed{2}\cdots$: 数字カードは、表面に自然数が書かれたカードである。
- **ダミーカード** $\boxed{}$: ダミーカードは、表面に何も書かれていないカードである。

本稿で使用するカードは、表面に異なる情報が記載されているが、裏面はいずれも $\boxed{?}$ であり、裏向きに置かれた状態では区別できない。表面に記号 a を持つ裏向きのカードを $\boxed{[a]}$ と表すことにする。これは便宜上の表記であり、実際の見た目は $\boxed{?}$ である。

2.2 整数符号化

整数符号化では、整数 x $(1 \le x \le k)$ を次のように表す。

$$E_k(x) = \underbrace{\boxed{\clubsuit}\cdots\boxed{\clubsuit}}_{x-1}\boxed{\heartsuit}\underbrace{\boxed{\clubsuit}\cdots\boxed{\clubsuit}}_{k-x}$$

すなわち、$\boxed{\heartsuit}$ が置かれた位置が整数 x を表す。

例として、$k=5$ の場合の整数符号化を以下に示す。

$$E_5(1) = \boxed{\heartsuit}\boxed{\clubsuit}\boxed{\clubsuit}\boxed{\clubsuit}\boxed{\clubsuit}$$
$$E_5(2) = \boxed{\clubsuit}\boxed{\heartsuit}\boxed{\clubsuit}\boxed{\clubsuit}\boxed{\clubsuit}$$
$$E_5(3) = \boxed{\clubsuit}\boxed{\clubsuit}\boxed{\heartsuit}\boxed{\clubsuit}\boxed{\clubsuit}$$
$$E_5(4) = \boxed{\clubsuit}\boxed{\clubsuit}\boxed{\clubsuit}\boxed{\heartsuit}\boxed{\clubsuit}$$
$$E_5(5) = \boxed{\clubsuit}\boxed{\clubsuit}\boxed{\clubsuit}\boxed{\clubsuit}\boxed{\heartsuit}$$

2.3 シャッフル

本節では、パイルスクランブルシャッフル（Pile-Scramble Shuffle：PSS）およびパイルシフティングシャッフル（Pile-Shifting Shuffle：PShift）について述べる。

2.3.1 パイルスクランブルシャッフル（PSS）

パイルスクランブルシャッフル（PSS）は、Ishikawa ら [3] によって提案された、同じカード枚数からなるカード束の列に対するシャッフル操作である。S_n を n 次対称群とする。同じ枚数からなる n 個のカード束に対し、一様ランダムな置換 $\pi \in S_n$ を適用することで、各カード束をランダムに並び替える。カード束列 $(p_1, p_2, \ldots, p_n)$ に対して PSS

© 2026 Information Processing Society of Japan

を適用すると、$(p_{\pi^{-1}(1)}, p_{\pi^{-1}(2)}, \ldots, p_{\pi^{-1}(n)})$ が得られる。PSS を適用することを $[\cdot|\cdot|\cdots|\cdot|\cdot]$ で表す。

例えば、$n = 3$ の場合、PSS を適用するとカード束列は次の 6 通りのいずれかが一様ランダムに選ばれる。

$$\left[\boxed{p_1}\middle|\boxed{p_2}\middle|\boxed{p_3}\right] \xrightarrow{\text{PSS}} \left\{\begin{array}{lll} \boxed{p_1}\boxed{p_2}\boxed{p_3} & \boxed{p_1}\boxed{p_3}\boxed{p_2} & \boxed{p_2}\boxed{p_1}\boxed{p_3} \\ \boxed{p_2}\boxed{p_3}\boxed{p_1} & \boxed{p_3}\boxed{p_1}\boxed{p_2} & \boxed{p_3}\boxed{p_2}\boxed{p_1} \end{array}\right.$$

2.3.2 パイルシフティングシャッフル（PShift）

パイルシフティングシャッフル（PShift）は、Shinagawa ら [7] によって提案された、同じ枚数からなる n 個のカード束に対し、一様ランダムな整数 $r \in \mathbb{Z}/n\mathbb{Z}$ を選択し、全てのカード束を右方向へ r 個だけ巡回移動させるシャッフル操作である。

例えば、3 個のカード束に対して PShift を適用すると、次の 3 通りのいずれかが一様ランダムに選ばれる。

$$\left\langle\boxed{p_1}\middle|\boxed{p_2}\middle|\boxed{p_3}\right\rangle \xrightarrow{\text{PShift}} \left\{\begin{array}{l} \boxed{p_1}\boxed{p_2}\boxed{p_3} \\ \boxed{p_3}\boxed{p_1}\boxed{p_2} \\ \boxed{p_2}\boxed{p_3}\boxed{p_1} \end{array}\right.$$

なお、カード束が 1 枚のみで構成される場合、PShift はランダムカット（Random Cut：RC）と呼ばれる。

2.4 バッチング技術

本研究では、複数回のシャッフル操作を同時に実行するため、既存研究で提案されたバッチング技術および巡回バッチング技術を用いる。これらの技術により、複数回実行されるシャッフルをまとめて処理できるため、シャッフル回数を削減できる。

2.4.1 バッチング

バッチングとは、並列に実行される複数回の PSS を追加カードを用いて 1 回の PSS で実行する技術である [8]。

各 PSS で用いられるカード束の先頭に識別用の二色カードを追加し、異なるシャッフル群を区別する。その後、全体に対して 1 回の PSS を適用し、最後に識別カードに従ってカード束を元のグループごとに並べ替えることで、各グループに対して独立な PSS を実現できる。

例えば、2 つの PSS (p_1, p_2), (q_1, q_2) を同時に実行する場合、各束にそれぞれ $\boxed{\heartsuit}$, $\boxed{\clubsuit}$ を識別カードとして追加する。

$$\begin{array}{cccc} \boxed{\heartsuit} & \boxed{\heartsuit} & \boxed{\clubsuit} & \boxed{\clubsuit} \\ \boxed{p_1} & \boxed{p_2} & \boxed{q_1} & \boxed{q_2} \end{array}$$

このカード列全体に対して 1 回の PSS を適用した後、識別カードを基準としてカード束を並べ替えることで、

$$\begin{array}{cccc} \boxed{\heartsuit} & \boxed{\heartsuit} & \boxed{\clubsuit} & \boxed{\clubsuit} \\ \boxed{p_{i_1}} & \boxed{p_{i_2}} & \boxed{q_{j_1}} & \boxed{q_{j_2}} \end{array}$$

が得られる。ここで、$(i_1, i_2), (j_1, j_2) \in S_2$ はそれぞれ独立な一様ランダム置換となる。

このように、追加カードを利用することで、複数回の PSS を 1 回の PSS で実現できる。

2.4.2 巡回バッチング

巡回バッチングとは、複数回の PShift を、2 回の PSS で同時に実行するための技術である [5,9]。

例として (p_1, p_2, p_3)、(p_4, p_5, p_6)、および (p_7, p_8) に対する 3 つの PShift を同時に実行する場合を考える。

(1) **1 回目のシャッフル**： 以下のカード列に対して、各列を 1 つの束とし、PSS を適用する。

$$\left[\begin{array}{cccccccc} \boxed{1} & \boxed{2} & \boxed{3} & \boxed{4} & \boxed{5} & \boxed{6} & \boxed{7} & \boxed{8} \\ \boxed{1} & \boxed{2} & \boxed{3} & \boxed{4} & \boxed{5} & \boxed{6} & \boxed{7} & \boxed{8} \end{array}\right]$$

これによって、数字カードの順序が

$$(1,2,3,4,5,6,7,8) \to (a_1, a_2, \ldots, a_8)$$

となったとする。これを 3 列、3 列、2 列のグループに分け、各グループの下段のカード列を巡回的に 1 個左にシフトすることで、以下のカード列が得られる。

$$\begin{array}{cccccccc} \boxed{a_1} & \boxed{a_2} & \boxed{a_3} & \boxed{a_4} & \boxed{a_5} & \boxed{a_6} & \boxed{a_7} & \boxed{a_8} \\ \boxed{a_2} & \boxed{a_3} & \boxed{a_1} & \boxed{a_5} & \boxed{a_6} & \boxed{a_4} & \boxed{a_8} & \boxed{a_7} \end{array}$$

上記のカード列は、3 個の巡回置換 (a_1, a_2, a_3), (a_4, a_5, a_6), (a_7, a_8) とみなせる。

(2) **2 回目のシャッフル**：シャッフルを適用したいカード列に対して、巡回置換と識別用カードを付加し、2 回目の PSS を適用する。

$$\left[\begin{array}{cccccccc} \boxed{p_1} & \boxed{p_2} & \boxed{p_3} & \boxed{p_4} & \boxed{p_5} & \boxed{p_6} & \boxed{p_7} & \boxed{p_8} \\ \boxed{a_1} & \boxed{a_2} & \boxed{a_3} & \boxed{a_4} & \boxed{a_5} & \boxed{a_6} & \boxed{a_7} & \boxed{a_8} \\ \boxed{a_2} & \boxed{a_3} & \boxed{a_1} & \boxed{a_5} & \boxed{a_6} & \boxed{a_4} & \boxed{a_8} & \boxed{a_7} \\ \boxed{1} & \boxed{1} & \boxed{2} & \boxed{3} & \boxed{3} & \boxed{3} & \boxed{4} & \boxed{4} \end{array}\right]$$

(3) **カードの並べ替え**： 巡回置換と識別用カードを公開する。以下の図のように、まず識別用カードごとに列を並び替え、次に巡回置換の対応関係を満たすように列を並び替える。これにより、カード列にはそれぞれ対応する巡回置換が適用される。

$$\begin{array}{cccccccc} \boxed{?} & \boxed{?} & \boxed{?} & \boxed{?} & \boxed{?} & \boxed{?} & \boxed{?} & \boxed{?} \\ \boxed{8} & \boxed{3} & \boxed{1} & \boxed{6} & \boxed{7} & \boxed{4} & \boxed{2} & \boxed{5} \\ \boxed{3} & \boxed{1} & \boxed{8} & \boxed{7} & \boxed{4} & \boxed{6} & \boxed{5} & \boxed{2} \\ \boxed{1} & \boxed{1} & \boxed{1} & \boxed{2} & \boxed{2} & \boxed{2} & \boxed{3} & \boxed{3} \end{array}$$

以上が巡回バッチングの手順である。このように、並列的に実行される複数回の PShift は、巡回バッチングを用いて 2 回の PSS に集約できる。

© 2026 Information Processing Society of Japan

2.5 問題の定義

本節では、マカロを数学的に定式化する。盤面 $u \times v$ における白マスの集合を $B \subseteq \{1,\ldots,u\} \times \{1,\ldots,v\}$ とする。白マスの集合 B は l 個の部屋 $R_1, R_2, \ldots, R_l$ に分割されている。各 R_i は連結した白マスの集合であり、異なる R_i, R_j は共通部分を持たず、$B = R_1 \cup R_2 \cup \cdots \cup R_l$ である。また、最大の部屋の大きさを $k = \max_i |R_i|$ 、すべての白マスの数を $n = |B|$、異なる部屋に属する隣接マスの組み合わせの数を t と表す。

一方、白マス以外のマスを黒マスと呼び、いくつかの黒マスには、上下左右のいずれかの方向を向く矢印が描かれている。これを矢印マスと呼ぶ。矢印マスの集合を $A \subseteq (\{1,\ldots,u\} \times \{1,\ldots,v\}) \setminus B$ とし、矢印の総数を $a = |A|$ とする。

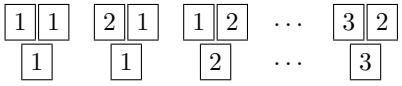
マカロの解とは、各白マスへの整数の割当て $(x_{(i,j)})_{(i,j)\in B}$（ただし $1 \le x_{(i,j)} \le k$）であって、部屋条件、隣接条件、矢印条件の 3 条件をすべて満たすものである。解が与えられたとき、r 番目の部屋の中で、整数 $x \in \{1,\ldots,|R_r|\}$ を持つマスの座標 (i,j) が一意に定まる。この座標を $f(r,x)$ で表すことにする。

3. 提案プロトコル 1

提案プロトコル 1 は、標準入力に対して 3 回のシャッフルを用いるプロトコルである。以下に提案プロトコル 1 を図 1 の問題例に適用した際の例とともにその手順を示す。

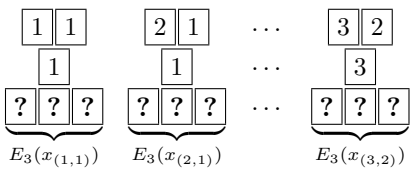
3.1 入力

(1) **準備**：すべての白マスについて、行番号と列番号を表す数字カードを 1 段目に、部屋番号を表す数字カードを 2 段目に、表向きに並べる。



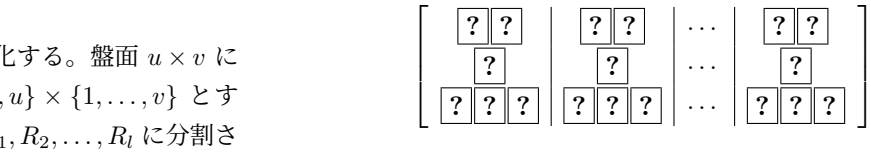
これらのカードを行カード、列カード、部屋カードと呼ぶことにする。

(2) **入力**：証明者 P は、各白マス (i,j) に対して、解となる整数符号化カード列 $E_k(x_{(i,j)})$ を 3 段目に裏向きに並べて入力を行う。

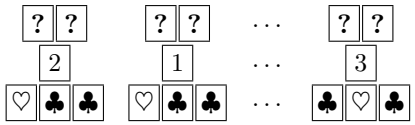


3.2 部屋条件の検証

(1) **シャッフル 1 回目**：すべてのカードを裏向きにし、カード列に対して PSS を実行する。



(2) **部屋条件の検証**：部屋カードと整数符号化カード列をすべて表にする。



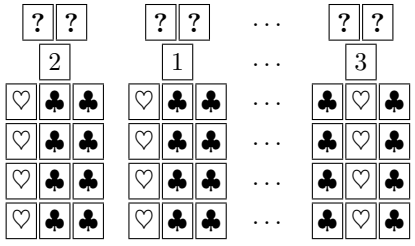
すべての部屋 r が以下の式を満たすことを確認する。

$$\{E_k(x_{(i,j)}) \mid (i,j) \in R_r\} = \{E_k(w) \mid 1 \le w \le |R_r|\}$$

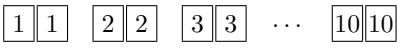
すべての部屋で条件を満たしている場合、次の手順に進み、そうでなければ検証者 V は証明を棄却する。

3.3 隣接条件と矢印条件の検証

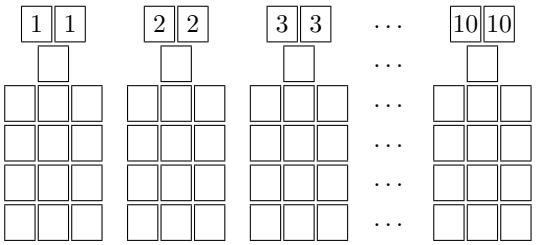
(1) **整数符号化カード列のコピー**：各整数符号化カード列を 4 つにコピーする。これは、1 つの数字マスについて検証対象となる隣接マスおよび矢印マスが最大 4 つ存在するためである。



(2) **矢印条件の検証準備**： 巡回バッチングを用いて矢印条件を検証するため、以下の図のように $(2k-1)a$ 束のカード束を作成する。なお、本稿の例（$k=3, a=2$）では $(2\times 3-1)\times 2 = 10$ 束を用意する。



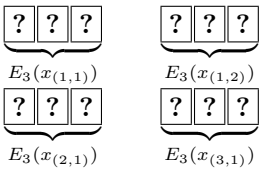
手順 (1) で作成したカード束とカード枚数を揃えるため、各束にダミーカード $(4k+1)$ 枚を追加する。



(3) **2 回目のシャッフル**：手順 (1) および手順 (2) で作成したカード束をすべて裏向きにし、まとめて PSS を適用する。シャッフル後、各カード束の種類を識別するため、各束の 2 段目にあるカード（部屋カードまたはダミーカード）を表にする。

© 2026 Information Processing Society of Japan

(4) **隣接条件の検証用カード列作成**：2 段目が部屋カードであったカード束について、行カードおよび列カードを公開する。異なる部屋に属する隣接マス $(i,j),(i',j')$ について、整数符号化カード列 $E_k(x_{(i,j)}), E_k(x_{(i',j')})$ を図のように縦に並べる。



(5) **矢印条件検証用カード列の作成**：2 段目がダミーカードであったカード束について、各束から 1 段目のカードのペアを裏面のまま取り出す。i 番目のペアの値を e_i とすると、$(e_1, e_2, \ldots, e_{(2k-1)a})$ の $(2k-1)a$ 個のペアが得られたことになる。$(2k-1)$ 個のペアを 1 つのグループとし a 個のグループを作成する。各グループに識別用カードを付加し、以下の図のように並べる。

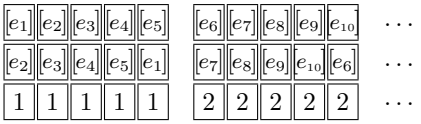
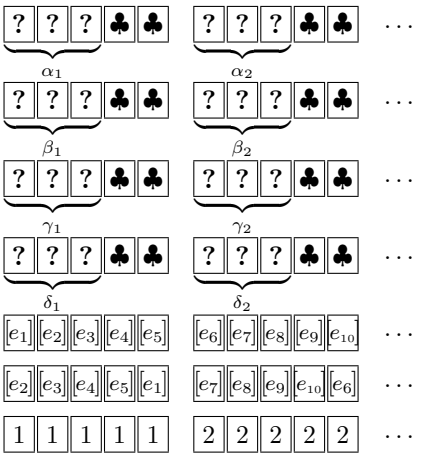


図 2 に示すように、i 番目の矢印が指すマスの整数符号化カード列を α_i、その他の周りのマスの整数符号化カード列を $\beta_i, \gamma_i, \delta_i$ とする。各矢印について、1 行目にカード列 α_i を、2〜4 行目にカード列 $\beta_i, \gamma_i, \delta_i$ をそれぞれ図のように $k-1$ 枚のクラブの追加カードと並べて配置し、作成したカード列の上に重ねて並べる。



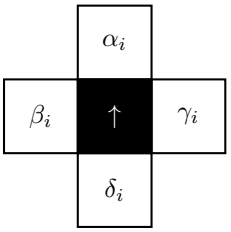
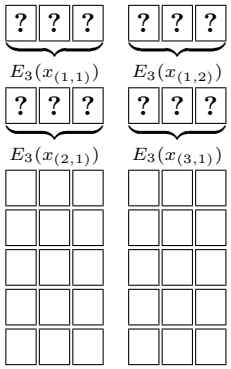
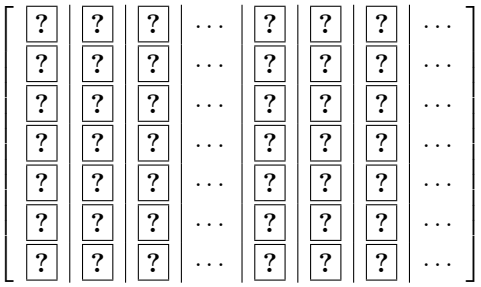


図 2: i 番目の矢印と周囲 4 マスの符号化カード列の配置

(6) **ダミーカードの追加**：手順 (4) のカード束に対して、手順 (5) のカード束のカード枚数と揃えるために、ダミーカードを図のように $5kt$ 枚追加する。



(7) **3 回目のシャッフル**：手順 (5) で作成したカード束と手順 (6) で作成したカード束を並べ、1 列ずつのカード束に対して PSS を行う。



(8) **隣接条件の検証**：手順 (7) の PSS の結果得られたカード列について、最下段の識別カード列を公開する。識別カードがダミーカードである列を抽出し、その列に対応する 1 行目および 2 行目のカードを公開する。検証者 V はハートが 2 枚以上含まれる列がないことを確認する。隣接条件を確認できなければ証明を棄却する。

(9) **矢印条件の検証**： 手順 (7) のシャッフルの結果、一番下の行が識別カードだった列を抽出して、5-7 行目をすべて表にし、巡回バッチングのカードの並び替えの手順を基に並び替える。1-4 行目には PShift による巡回置換後のカード列が得られる。この結果を用いて、既存プロトコルと同様の手順で各矢印について矢印条件を検証する。V はそれぞれの矢印について矢印条件を満たしていることを確認する。確認できなければ証明を棄却する。

3.4 カード枚数とシャッフル回数

提案プロトコル 1 で用いるカードは、入力カード kn 枚と追加カード $(3k+3)n + 8a(k^2+k-1) + 5kt$ 枚の計 $(4k+3)n + 8a(k^2+k-1) + 5kt$ 枚である。また、必要なシャッフル回数は計 3 回である。

追加カードの内訳は、3.1 節で $3n$ 枚、3.3 節の手順 (1) で $3kn$ 枚、手順 (2) で $a(2k-1)(4k+3)$ 枚、手順 (5) で $a(6k-5)$ 枚、および 3.3 節の手順 (6) で $5kt$ 枚である。

© 2026 Information Processing Society of Japan

3.5 提案プロトコル 1 の正当性

本節では、提案プロトコル 1 が完全性、健全性、ゼロ知識性を満たすことを示す。

3.5.1 完全性

入力カード列がパズルの正しい解に対応していると仮定する。以下では、正しい解が各制約を満たしていることから、いずれの検証手順においても棄却されることなく、V によって必ず受理されることを示す。まず、正しい解においては各部屋に 1 から部屋サイズまでの整数がちょうど 1 回ずつ現れるため、3.2 節の部屋条件の検証において棄却されることなく次の手順へ進む。次に、隣接するマスに同一の数字が配置されないことから、3.3 節手順 (8) の隣接条件の検証においても棄却されることはない。さらに、矢印の先の数字が周囲の数字より大きいという条件も充足されるため、3.3 節手順 (9) の矢印条件の検証においても棄却されない。以上より、入力カード列が正しい解に対応している場合、すべての検証手順を棄却されることなく通過し、最終的に受理されるため、提案プロトコルは完全性を満たす。

3.5.2 健全性

入力カード列がパズルの正しい解に対応していないと仮定する。このとき、入力カード列は「部屋条件」「隣接条件」「矢印条件」の少なくともいずれか 1 つの制約に違反している。まず、ある部屋において数字の重複や欠落が存在する場合、3.2 節の部屋条件の検証において V によって棄却される。次に、隣接するマスに同一の数字が配置されている場合、3.3 節手順 (8) の隣接条件の検証において棄却される。さらに、矢印の先の数字が周囲の数字より大きくない配置が含まれる場合、3.3 節手順 (9) の矢印条件の検証においても棄却される。以上より、入力カード列が正しい解に対応していない場合、いずれかの検証手順において必ず棄却されるため、提案プロトコルは健全性を満たす。

3.5.3 ゼロ知識性

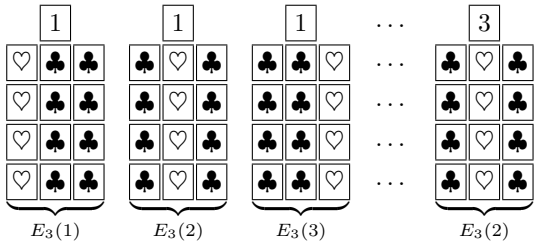
提案プロトコルにおいて裏向きのカードをめくる手順は、3.2 節の手順 (2) と 3.3 節の手順 (3) と手順 (8) と手順 (9) の 4 箇所である。ゼロ知識性を示すためには、これらのカードをめくった際に得られる情報は、解を知らなくても生成できることを示せばよい。3.2 節の手順 (2) では、各部屋番号ごとに 1 から $|R_r|$ の数字が重複なく現れるため、この情報は解を知らなくても生成できる。3.3 節の手順 (3) では、各部屋番号ごとに 1 から $|R_r|$ の数字と $a(2k-1)$ 枚のダミーカードが現れるため、この情報は解を知らなくても生成できる。3.3 節の手順 (8) では、各列の 1 行目と 2 行目が $\boxed{\heartsuit}\boxed{\clubsuit}$ または $\boxed{\clubsuit}\boxed{\heartsuit}$ である列はそれぞれ B 個であり、それ以外はすべて $\boxed{\clubsuit}\boxed{\clubsuit}$ であるため、この情報も解を知らなくても生成できる。3.3 節の手順 (9) は巡回バッチングを用いること以外は既存プロトコルと同様の手順であるため、これらの手順で得られる情報も解を知らなくても生成できる。以上より、提案プロトコルはゼロ知識性を満たす。

4. 提案プロトコル 2

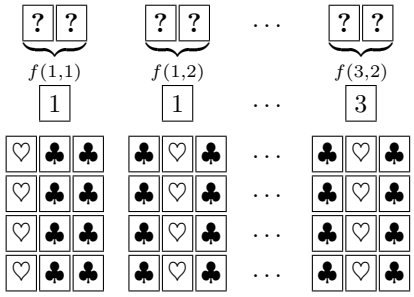
提案プロトコル 2 は座標入力のシャッフル 2 回のプロトコルである。以下に提案プロトコル 2 を図 1 に適用した際の例とともにその手順を示す。

4.1 入力

(1) **準備**： 部屋番号が r の部屋に対して、部屋カード $\boxed{r}$ と解に対応するカード列 $E_k(i)$ $(i = 1, \ldots, |R_r|)$ をそれぞれ 4 つずつ置く。

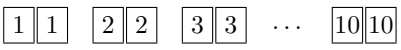


(2) **入力**:P はそのようなカード列に対して解となる行カード $\boxed{i}$ と列カード $\boxed{j}$ を裏向きに入力を行う。

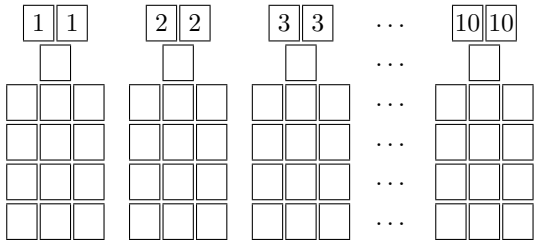


4.2 部屋条件の検証と矢印条件の検証準備

(1) **矢印条件の検証準備**: 提案プロトコル 1 と同様に、巡回バッチングを用いて矢印条件を検証するため、以下の図のように $(2k-1)a$ 束のカード束を作成する。



4.1 節手順 (2) で作成したカード束とカード枚数を揃えるため、各束にダミーカード $(4k+1)$ 枚を追加する。



(2) **1 回目のシャッフル**： 手順 (1) と 4.1 節手順 (2) で作成したカード束をすべて裏向きにし、まとめて PSS を適用する。シャッフル後、各カード束の種類を識別するため、各束の 2 段目にあるカード（部屋カードまた

© 2026 Information Processing Society of Japan

はダミーカード）を表にする。

(3) **部屋条件の検証**： 2 段目が部屋カードであったカード束について行カード $\boxed{i}$、列カード $\boxed{j}$ を表にし、それぞれの整数符号化カード列をパズルの盤面のマス (i, j) に置いていく。すべての整数符号化カード列を置いたとき、V は各白マスに一つずつ整数符号化カード列が置かれていることを確認する。確認できなければ証明を棄却する。

(4) **矢印条件検証用カード列の作成**：2 段目がダミーカードであったカード束について、各束から 1 段目のカードのペアを裏面のまま取り出す。i 番目のペアの値を e_i とすると、$(e_1, e_2, \ldots, e_{(2k-1)a})$ の $(2k-1)a$ 個のペアが得られたことになる。$(2k-1)$ 個のペアを 1 つのグループとし a 個のグループを作成する。各グループに識別用カードを付加し、以下の図のように並べる。

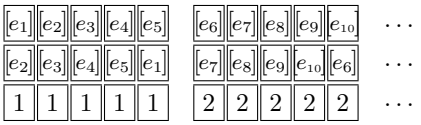
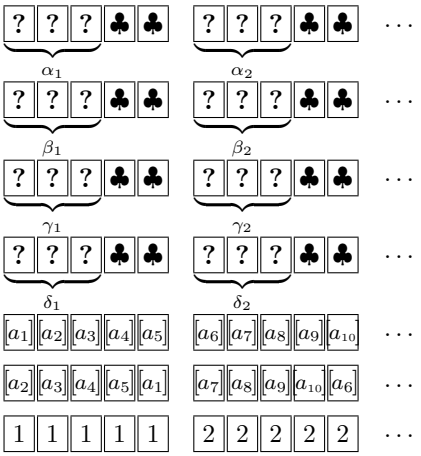
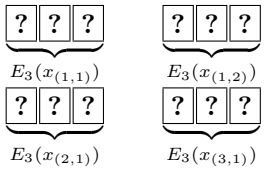


図 2 に示すように、i 番目の矢印が指すマスの整数符号化カード列を α_i、その他の周りのマスの整数符号化カード列を $\beta_i, \gamma_i, \delta_i$ とする。各矢印について、1 行目にカード列 α_i を、2〜4 行目にカード列 $\beta_i, \gamma_i, \delta_i$ をそれぞれ配置し、作成したカード列の上に重ねて並べる。

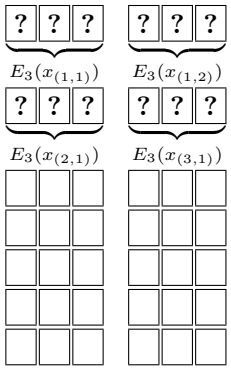


4.3 隣接条件と矢印条件の検証

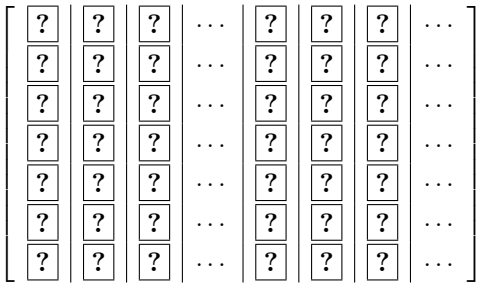
(1) **隣接条件の検証用カード列作成**：2 段目が部屋カードであったカード束について、行カードおよび列カードを公開する。異なる部屋に属する隣接マス $(i, j), (i', j')$ について、整数符号化カード列 $E_k(x_{(i,j)}), E_k(x_{(i',j')})$ を図のように縦に並べる。



(2) **ダミーカードの追加**：このカード束に対して、4.2 節手順 (1) のカード束のカード枚数と揃えるために、ダミーカードを図のように $5kt$ 枚追加する。



(3) **2 回目のシャッフル**：4.2 節手順 (1) で作成したカード束と手順 (2) で作成したカード束を裏向きに並べ、1 列ずつのカード束に対して PSS を行う。



(4) **隣接条件の検証**：手順 (3) の PSS の結果得られたカード列について、最下段の識別カード列を公開する。識別カードがダミーカードである列を抽出し、その列に対応する 1 行目および 2 行目のカードを公開する。検証者 V はハートが二枚以上含まれる列がないことを確認する。隣接条件を確認できなければ証明を棄却する。

(5) **矢印条件の検証**手順 (3) のシャッフルの結果、一番下の行が識別カードだった列を抽出して、5-7 行目をすべて表にし、巡回バッチングの手順を基に並び替える。1-4 行目には PShift による巡回置換後のカード列が得られる。この結果を用いて、既存プロトコルと同様の手順で各矢印について矢印条件を検証する。V はそれぞれの矢印について矢印条件を満たしていることを確認する。確認できなければ証明を棄却する。

4.4 カード枚数とシャッフル回数

提案プロトコル 2 で用いるカードは、入力カード $2n$ 枚と追加カード $(4k+1)n + 8a(k^2+k-1) + 5kt$ 枚の計 $(4k+3)n + 8a(k^2+k-1) + 5kt$ 枚である。また、必要なシャッフル回数は計 2 回である。

追加カードの内訳は、4.1 節の手順 (1) で $(4k+1)n$ 枚、4.2 節の手順 (1) で $a(2k-1)(4k+3)$ 枚、4.3 節の手順 (1) で $a(6k-5)$ 枚、および 4.3 節の手順 (2) で $5kt$ 枚である。

© 2026 Information Processing Society of Japan

4.5 提案プロトコル 2 の正当性

本節では、提案プロトコル 2 が完全性、健全性、ゼロ知識性を満たすことを示す。

4.5.1 完全性

入力カード列がパズルの正しい解に対応していると仮定する。以下では、正しい解が各制約を満たしていることから、いずれの検証手順においても棄却されることなく、V によって必ず受理されることを示す。まず、正しい解においては各部屋に 1 から部屋サイズまでの整数がちょうど 1 回ずつ現れるため、4.2 節の部屋条件の検証において棄却されることなく次の手順へ進む。次に、隣接するマスに同一の数字が配置されないことから、4.3 節手順 (4) の隣接条件の検証においても棄却されることはない。さらに、矢印の先の数字が周囲の数字より大きいという条件も充足されるため、4.3 節手順 (5) の矢印条件の検証においても棄却されない。以上より、入力カード列が正しい解に対応している場合、すべての検証手順を棄却されることなく通過し、最終的に受理されるため、提案プロトコルは完全性を満たす。

4.5.2 健全性

入力カード列がパズルの正しい解に対応していないと仮定する。このとき、入力カード列は「部屋条件」「隣接条件」「矢印条件」の少なくともいずれか 1 つの制約に違反している。まず、ある部屋において数字の重複や欠落が存在する場合、4.2 節の手順 (3) の部屋条件の検証において V によって棄却される。次に、隣接するマスに同一の数字が配置されている場合、4.3 節手順 (4) の隣接条件の検証において棄却される。さらに、矢印の先の数字が周囲の数字より大きくない配置が含まれる場合、4.3 節手順 (5) の矢印条件の検証においても棄却される。以上より、入力カード列が正しい解に対応していない場合、いずれかの検証手順において必ず棄却されるため、提案プロトコルは健全性を満たす。

4.5.3 ゼロ知識性

提案プロトコルにおいて裏向きのカードをめくる手順は、4.2 節の手順 (2) と手順 (3)、4.3 節の手順 (4) と手順 (5) の 4 箇所である。ゼロ知識性を示すためには、これらのカードをめくった際に得られる情報は、解を知らなくても生成できることを示せばよい。4.2 節の手順 (2) では、各部屋番号ごとに 1 から $|R_r|$ の数字と $(2k-1)a$ 枚のダミーカードが現れるため、この情報は解を知らなくても生成できる。4.2 節の手順 (3) では、すべての白マスの座標が重複なく現れるため、この情報は解を知らなくても生成できる。4.3 節の手順 (4) では、各列の 1 行目と 2 行目が $\boxed{\heartsuit}\boxed{\clubsuit}$ または $\boxed{\clubsuit}\boxed{\heartsuit}$ である列はそれぞれ t 個であり、それ以外はすべて $\boxed{\clubsuit}\boxed{\clubsuit}$ であるため、この情報も解を知らなくても生成できる。4.3 節の手順 (5) は巡回バッチングを用いること以外は既存プロトコルと同様の手順であるため、これらの手順で得られる情報も解を知らなくても生成できる。以上より、提案プロトコルはゼロ知識性を満たす。

5. おわりに

本稿では、マカロに対するカードベースゼロ知識証明として、盤面サイズに依存しない定数回のシャッフルで検証可能な 2 種類のプロトコルを提案した。提案プロトコル 1 では 3 回、提案プロトコル 2 では 2 回のシャッフルで検証でき、いずれも完全性、健全性、ゼロ知識性を満たすことを示した。なお、異なる枚数のカード束を同時にシャッフルできる一般化パイルスクランブルシャッフル [4] を用いることで、どちらのプロトコルも使用するダミーカードの $a(2k-1)(4k+1)+5kt$ 枚を省略できる。

謝辞 本研究は JSPS 科研費 JP21K17702, JP26K21162 と JST CREST JPMJCR22M1 の支援を受けた。

参考文献

[1] X. Bultel, J. Dreier, J. Dumas, P. Lafourcade, D. Miyahara, T. Mizuki, A. Nagao, T. Sasaki, K. Shinagawa, and H. Sone. Physical zero-knowledge proof for Makaro. In T. Izumi and P. Kuznetsov eds., *Stabilization, Safety, and Security of Distributed Systems*, Vol. 11201 of *LNCS*, pp. 111–125. Springer, 2018.

[2] S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof-systems (extended abstract). In *Proceedings of the 17th Annual ACM Symposium on Theory of Computing*, pp. 291–304, 1985.

[3] R. Ishikawa, E. Chida, and T. Mizuki. Efficient card-based protocols for generating a hidden random permutation without fixed points. In C. S. Calude and M. J. Dinneen eds., *Unconventional Computation and Natural Computation*, Vol. 9252 of *LNCS*, pp. 215–226, Cham, 2015. Springer.

[4] Y. Ishizaki, K. Shinagawa, and K. Nuida. Batching techniques using generalized pile-scramble shuffles and branching shuffles. *IEICE Trans. Fundam.*, 2026. In Press.

[5] K. Miyamoto and K. Shinagawa. Graph automorphism shuffles from pile-scramble shuffles. *New Gener. Comput.*, 40:199–223, 2022.

[6] Nikoli. マカロの遊び方、ルール、解き方.

[7] K. Shinagawa, T. Mizuki, J. Schuldt, K. Nuida, N. Kanayama, T. Nishide, G. Hanaoka, and E. Okamoto. Card-based protocols using regular polygon cards. *IEICE Trans. Fundam.*, E100.A(9):1900–1909, 2017.

[8] K. Shinagawa and K. Nuida. A single shuffle is enough for secure card-based computation of any Boolean circuit. *Discrete Applied Mathematics*, 289:248–261, 2021.

[9] 本間日綺, 品川和雅. Str8ts に対するカードベースゼロ知識証明, 2025.

© 2026 Information Processing Society of Japan